

Lab-02 Report: Generated 2026-09-10 12:29

Names:

Group Name:

- **Q1.1:** Notice above that some features are continuous, whereas some are discrete (limited to a few values). Pay attention to our target features, `Heating_Load` and `Cooling_Load`. Are these variables unimodal (one major peak) or bimodal (two major peaks)? What might this tell us about the building types in the dataset?
- **A1.1:** They are generally bimodal. This tells us that the data set covers buildings that fall into roughly two different buckets of size (smaller and larger)
- **Q1.2:** Note above the significant differences in scale between the values of `Surface_Area` and `Relative_Compactness`. Why would feeding these features directly into a machine learning model be problematic? What type of preprocessing step could we introduce to help introduce this issue?
- **A1.2:** This would be problematic because the differences in scale would create large gradients that would negatively impact training because the gradients would not be proportional. We can solve this via normalization!
- **Q1.3:** Between the scatter and correlation matrices, you can clearly see how different features are related to one another. What input features do you expect to have the largest impact (with resulting weights and biases) on `Heating_Load` and `Cooling_Load`? Which single feature seems to have the strongest relationship with heating and cooling loads?
- **A1.3:** Likely `Overall_Height` has the largest impact as it has the highest correlation. Second would be roof area as it has the highest negative correlation and would push weights in the opposite direction than `overall_height`.
- **Q1.4:** In your own words, summarize what have you learned about this dataset from this initial exploration and visualization? In what ways is this dataset

representative of buildings and energy efficiency in general, and in what ways is it not representative?

- **A1.4):** There is a large grouping between smaller buildings and larger buildings, which would be representative of those categories. This dataset also shows how much height and roof area impact heating and cooling. This doesn't represent all buildings efficiency perfectly as it doesn't appear to account for wall thickness or building materials (glass buildings will be less efficient than a foam house for example).
- **Q1.5):** Conceptually, how have we transformed the inputs features (e.g. $\mathbf{X}$)? Why is this feature scaling important for gradient descent in linear regression? What might happen if features have very different scales? Answer briefly below, and use the subsequent code block to explore how two of the features in `X_scaled` are visually different than its equivalent feature in `X_raw`. To do this, chose two features from the dataframe and create a `matplotlib` figure with subplots for each, plotting both the scaled and raw data for each feature to compare.
- **A1.5):** We have effectively shifted the data to be closer together, they are now effectively centralized to 0 or near 0. The standard deviations have been also shifted and scaled down so one datapoint isn't super overblown. You can see that in the figure that while the data graphs appear the same for Surface area and Relative Compactness the x axis shifts so the average is around 0.

```
# Code for Q1.2 here. Visually compare the distributions of two of the raw and
scaled features.
# YOUR CODE STARTS HERE
# Two bar graphs comparing the raw vs normalized/scaled data

features_to_compare = ['Surface_Area', 'Relative_Compactness']

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

for row_idx, feature in enumerate(features_to_compare):
    # Left subplot: Raw distribution
    ax_raw = axes[row_idx, 0]
    ax_raw.hist(X_raw[feature], bins=20, color='tab:blue', alpha=0.7,
edgecolor='black')
    ax_raw.axvline(X_raw[feature].mean(), color='red', linestyle='--',
label=f'Mean = {X_raw[feature].mean():.2f}')
    ax_raw.set_title(f'Raw {feature}')
    ax_raw.set_xlabel('Value')
    ax_raw.set_ylabel('Count')
    ax_raw.legend()
```

```

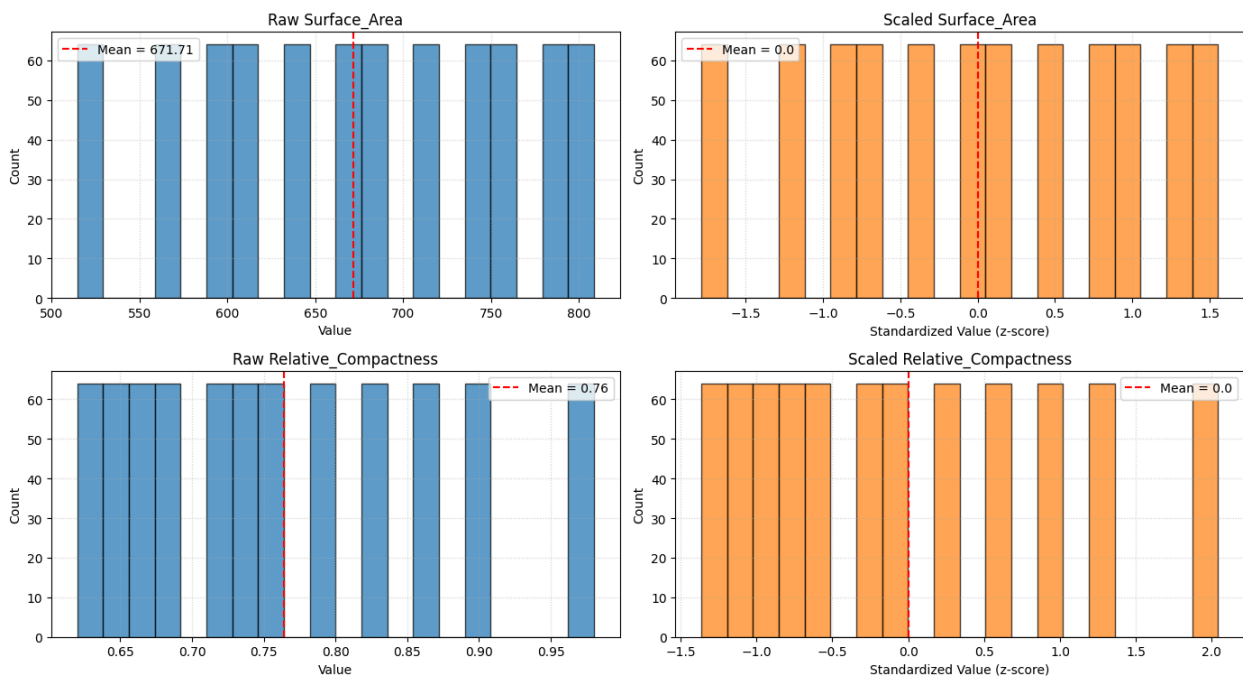
ax_raw.grid(True, linestyle=':', alpha=0.5)

# Right subplot: Scaled distribution
ax_scaled = axes[row_idx, 1]
ax_scaled.hist(X_scaled[feature], bins=20, color='tab:orange', alpha=0.7,
edgecolor='black')
ax_scaled.axvline(0, color='red', linestyle='--', label='Mean = 0.0')
ax_scaled.set_title(f'Scaled {feature}')
ax_scaled.set_xlabel('Standardized Value (z-score)')
ax_scaled.set_ylabel('Count')
ax_scaled.legend()
ax_scaled.grid(True, linestyle=':', alpha=0.5)

plt.suptitle('Q1.5: Raw vs. Scaled Distributions Comparison', fontsize=14,
y=1.02)
plt.tight_layout()
plt.show()
# YOUR CODE ENDS HERE

```

Q1.5: Raw vs. Scaled Distributions Comparison



```

class BuildingEnergyEfficiencyData(d2l.DataModule): #@save
    """Building Energy Efficiency data for linear regression."""
    def __init__(self, building_energy_df, train_val_split=0.8,
batch_size=32):
        super().__init__()
        self.save_hyperparameters()
        # Process energy_df similar as in part 1 when it was called df1
        # print(building_energy_df.info)
        # The resulting self.X and self.y need to be torch.tensors, so be sure
        to convert from pandas DataFrame to numpy array to torch tensor
        # .values gets the underlying numpy array from the DataFrame, and
        .to(torch.float32) converts to float32 tensor

```

```

        # example: tensor_vals =
torch.tensor(dataframe[column_name].values).to(torch.float32)

        X_raw = building_energy_df.drop(columns=['Heating_Load'])
        #, 'Cooling_Load'
        means = X_raw.mean() # column means
        stds = X_raw.std(ddof=0) # population std (ddof=0 like
StandardScaler)
        safe_stds = stds.replace(0, 1.0) # Protect against zero variance to
avoid divide-by-zero
        X_scaled = X_raw.copy()
        X_scaled = (X_scaled - means) / safe_stds

        self.data = torch.tensor(building_energy_df.values).to(torch.float32)
# get all data into one torch tensor
        self.X = torch.tensor(X_scaled.values).to(torch.float32) # select all
but the target prediction
        self.y =
torch.tensor(building_energy_df[['Heating_Load']].values).to(torch.float32) #
select the target prediction
        #print(self.x)
        total = len(self.data) # number of rows in our dataset
        self.num_train = int(total * train_val_split) # get just the amount we
will use for training (~80%)
        self.num_val = total - self.num_train # get the remaning elements
(validation set)
        print(self.num_train)
        #print(self.data)

# YOUR CODE STARTS HERE
@d2l.add_to_class(BuildingEnergyEfficiencyData)
def train_dataloader(self):
    return get_dataloader(self, True)

@d2l.add_to_class(BuildingEnergyEfficiencyData)
def val_dataloader(self):
    return get_dataloader(self, False)
# YOUR CODE ENDS HERE

@d2l.add_to_class(BuildingEnergyEfficiencyData) #@save
def get_dataloader(self, train):
    i = slice(0, self.num_train) if train else slice(self.num_train, None)
    print(i)
    return self.get_tensorloader((self.X, self.y), train, i)

```

- **Q2.1):** The implementation of `__init__()` above is incomplete, as we have to provide appropriate values for `num_train`, `num_val`, and set `X` and `y` accordingly. Implement this block and in the space below, in your own words, provide an explanation of what each line you added is doing and why. Note that this step should primarily be reusing code from Part 1 of this lab - the

`__init()` takes in the pandas dataframe from our CSV file and is performing the same steps! Note the comment above that we need to be working with pytorch [tensors](#) as opposed to our pandas and numpy objects we've been using so far this lab.

- **A2.1):** Done! See comments for explanations. We did norm like suggested
- **Q2.2):** Plot a histogram of any of the input features returned after calling `.train_dataloader()`. Compare to the previously plotted histograms of the corresponding feature from the initial exploratory data analysis. The graphs should look similar but not identical. Explain two key differences between the histograms and their cause.
- **A2.2):** Okay, so one major change is that now it is roughly centered on zero (normalized). Another thing is that the values appear to be more continuous rather than discrete, which is because we are sampling one training batch (32 samples) that are also normalized. If you sum up the columns it appears that they sum to 32 which is correct!

```
building_energy_df = pd.read_csv('data/building_energy_efficiency_data.csv')
BuildingEnergy = BuildingEnergyEfficiencyData(building_energy_df)  # This
will otherwise use the default parameters for the init() of
EnergyEfficiencyData()

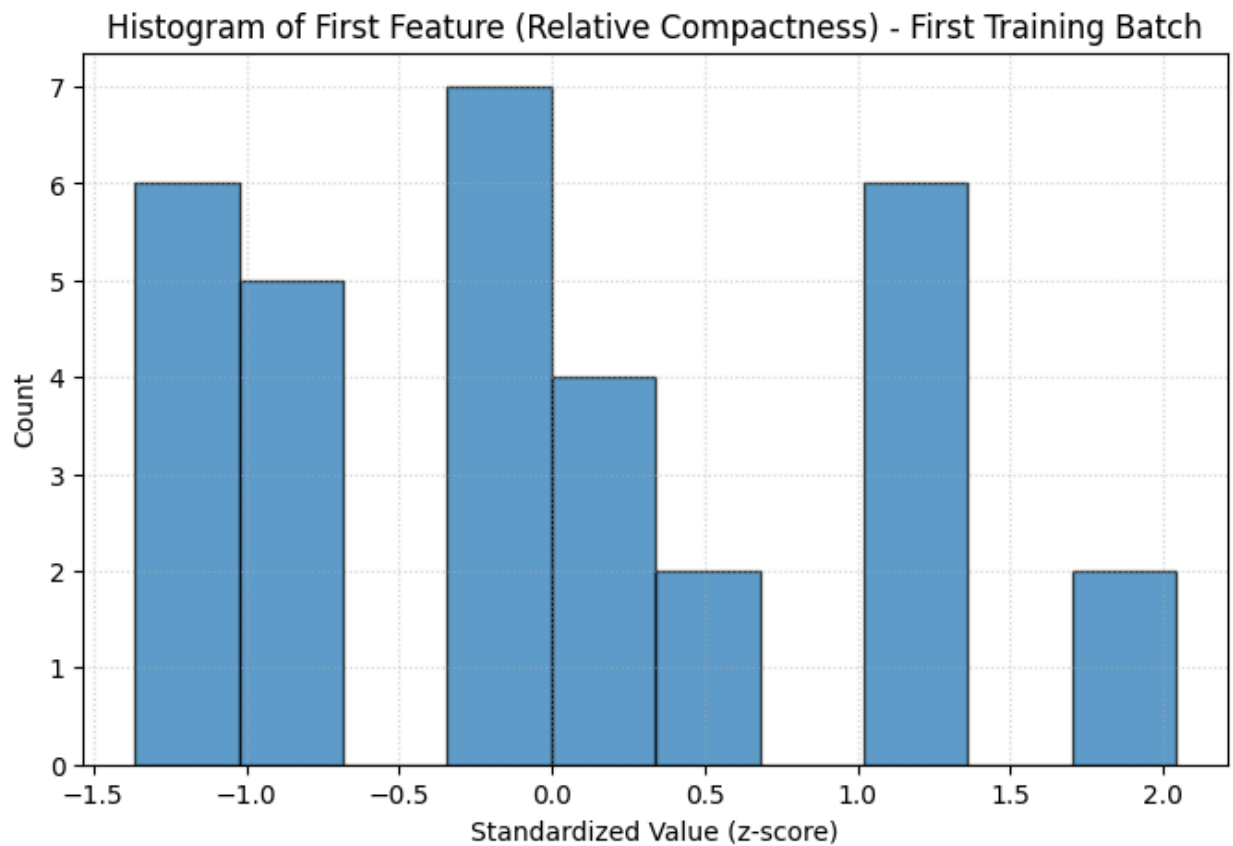
# Use iter() to turn the dataloader generator into an iterator, then use
next() to get the next batch
X, y = next(iter(BuildingEnergy.train_dataloader()))

# Plot this histogram of the first feature in the first training batch

# YOUR CODE STARTS HERE
feature_0 = X[:, 0].numpy()
# Plot the histogram
plt.figure(figsize=(8, 5))
plt.hist(feature_0, bins=10, edgecolor='black', color='tab:blue', alpha=0.7)
plt.title('Histogram of First Feature (Relative Compactness) - First Training
Batch')
plt.xlabel('Standardized Value (z-score)')
plt.ylabel('Count')
plt.grid(True, linestyle=':', alpha=0.5)
plt.show()
# YOUR CODE ENDS HERE
```

614

slice(0, 614, None)



- **Q3.1):** What are the dimensions for $\mathbf{w}$ (`self.w`) and b (`self.b`)? Why did we need to consider the number of input features when initializing this model?
- **A3.1):** The dim for weights is (9, 1) which is correct as we have 9 input features. It is important to track that as we may need to transpose the weights depending on how we do the forward pass. The bias is dim (1) which is expected as its just one value.
- **Q3.2):** If we wanted this model to predict both `Heating_Load` and `Cooling_Load`, how would our dataloader change, and what would need to change in our model implementation above? Be specific.
- **A3.2):** Our dataloader would need to init differently and pull both `Heating_Load` and `Cooling_Load` into the output `y`. The shape of the weights would also change as it would go from (9,1) to (8,1) as we would have 1 less input feature (probably)

```
# Grab a batch to determine the number of input features
X, y = next(iter(BuildingEnergy.train_dataloader()))
num_inputs = X.shape[1]
```

```

BuildingModel = LinearRegression_v1(num_inputs, lr=0.03)

# YOUR CODE STARTS HERE
print("I AM AGI, THESE ARE MY WEIGHT (shape): ", BuildingModel.w.shape)
print("I AM AGI, THESE ARE MY BIAS (shape): ", BuildingModel.b.shape)

# YOUR CODE ENDS HERE

slice(0, 614, None)
I AM AGI, THESE ARE MY WEIGHT (shape):  torch.Size([9, 1])
I AM AGI, THESE ARE MY BIAS (shape):  torch.Size([1])

```

- **Q3.3):** Does our `BuildingModel` make a good prediction of $\hat{y}$ (the Heating_Load)? Why or why not? Explain your reasoning, and use the following code block to visualize your answer. One approach would be to visualize your answer by plotting the residuals of `y_hat` compared to `y`. Note that `matplotlib` does not plot tensors directly - convert any values to `numpy` arrays first.
- **A3.3):** No! It is not even close. It appears that all of our predictions end up being 0. We haven't done any gradient descent (training) so the weights are currently set to the default and are not fitted to the data.

```

# Determine the accuracy of the model's initial prediction

# YOUR CODE STARTS HERE
# 1. Compute predictions
y_hat = BuildingModel.forward(X)

# 2. Convert to 1D numpy arrays
y_actual = y[:, 0].detach().cpu().numpy().flatten() if (y.ndim > 1 and
y.shape[1] > 1) else y.detach().cpu().numpy().flatten()
y_pred = y_hat.detach().cpu().numpy().flatten()
sample_indices = np.arange(len(y_actual))

# 3. Plot both on the same graph
plt.figure(figsize=(12, 5))

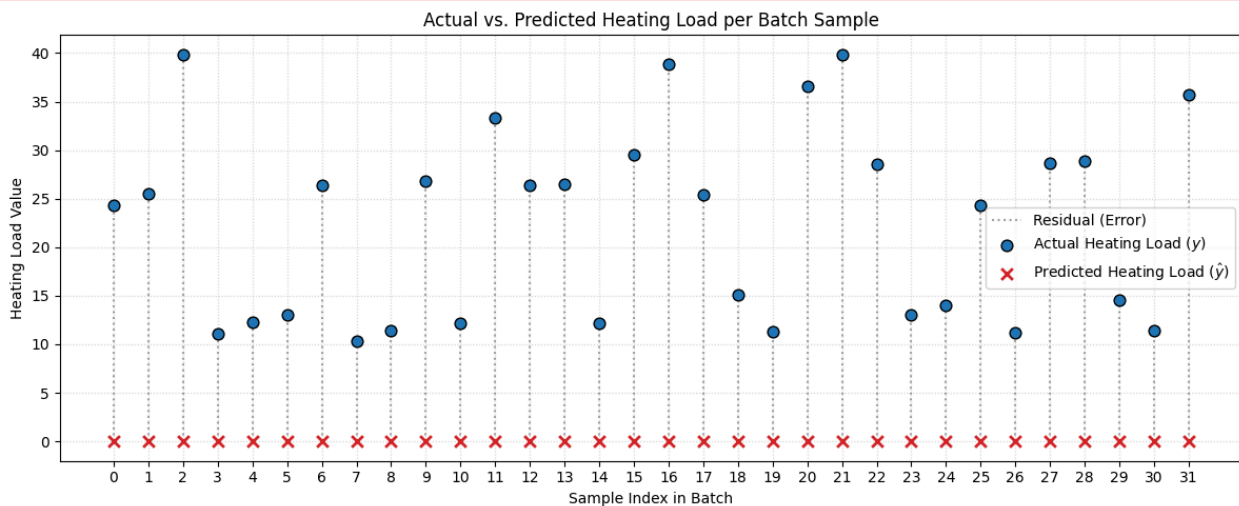
# Vertical dotted lines connecting actual to predicted (shows the error for
each sample)
plt.vlines(sample_indices, y_pred, y_actual, color='gray', linestyle=':',
alpha=0.7, label='Residual (Error)')

# Actual vs Predicted markers
plt.scatter(sample_indices, y_actual, color='tab:blue', s=60, edgecolors='k',
label='Actual Heating Load ($y$)', zorder=3)
plt.scatter(sample_indices, y_pred, color='tab:red', marker='x', s=60,
linewidths=2, label='Predicted Heating Load ($\hat{y}$)', zorder=3)

```

```
plt.title('Actual vs. Predicted Heating Load per Batch Sample')
plt.xlabel('Sample Index in Batch')
plt.ylabel('Heating Load Value')
plt.xticks(sample_indices)
plt.legend()
plt.grid(True, linestyle=':', alpha=0.5)
plt.tight_layout()
plt.show()
# YOUR CODE ENDS HERE
```

```
<>:20: SyntaxWarning: invalid escape sequence '\h'
<>:20: SyntaxWarning: invalid escape sequence '\h'
/tmp/ipykernel_28095/2306209211.py:20: SyntaxWarning: invalid escape sequence '\h'
plt.scatter(sample_indices, y_pred, color='tab:red', marker='x', s=60,
linewidths=2, label='Predicted Heating Load ( $\hat{y}$ )', zorder=3)
```



- **Q3.4):** Implement `loss()` below, and briefly explain your code for this part.
- **A3.4):** We basically did the first squared error equation. This is just 0.5 multiplied by the square of $y_{\text{hat}} - y$. The reshape is a safety guard and the `.mean()` is needed for the average loss across the batch.

```
@d2l.add_to_class(LinearRegression_v1) #@save
def loss(self, y_hat, y):
    # YOUR CODE STARTS HERE
    #print(0.5 * (torch.square(y_hat - y)))
    return (0.5 * (torch.square(y_hat - y.reshape(y_hat.shape)))) .mean()
    # YOUR CODE ENDS HERE
```

- **Q3.5):** Review the code block below, and describe in your own words, where the following steps happen:
a) Where the batch of training data is loaded

- b) Where the forward pass of the model is calculated
 - c) Where the loss is calculated
 - d) Where the gradients are calculated
 - e) Where the model parameters are updated
- **A3.5):** See code comments below.
 - **Q4.1):** In the code block below, use function `average_loss()` to compare the model's performance on both the training and validation data. Analyze the result: in your own words, how does the model perform? Are we underfitting or overfitting? How does this loss correspond to the real-world data we are attempting to collect (expected heating load efficiency of a building)?
 - **A4.1):** It appears our model is well-fitted as the predictions don't stray too far from our ideal $y_{\text{hat}} = y$ but is still not that accurate. It is about as good as our basic linear regression model can likely get. Our Validation loss is also lower than our training loss, so it's not overfitted potentially some light underfitting.

```

train_loss = average_loss(BuildingModel, BuildingEnergy.train_dataloader(),
BuildingModel.loss)

# YOUR CODE STARTS HERE
val_loss = average_loss(BuildingModel, BuildingEnergy.val_dataloader(),
BuildingModel.loss)
print(f"Training Loss (MSE / 2): {train_loss:.4f}")
print(f"Validation Loss (MSE / 2): {val_loss:.4f}")
# 2. Visualize performance on both training and validation sets
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
# Subplot 1: Average Loss comparison bar chart
bars = axes[0].bar(['Training Loss', 'Validation Loss'], [train_loss,
val_loss], color=['tab:blue', 'tab:orange'], width=0.45)
axes[0].set_ylabel('Average Loss (MSE / 2)')
axes[0].set_title('Average Loss: Training vs. Validation')
axes[0].grid(axis='y', linestyle=':', alpha=0.6)
# Annotate bars with numeric loss values
for bar in bars:
    h = bar.get_height()
    axes[0].text(bar.get_x() + bar.get_width() / 2.0, h, f'{h:.4f}',
ha='center', va='bottom', fontsize=10, fontweight='bold')
# Subplot 2: Predictions vs. Actuals parity plot
BuildingModel.eval()
with torch.no_grad():
    y_train_actual =
BuildingEnergy.y[:BuildingEnergy.num_train].cpu().numpy().flatten()

```

```

y_train_pred =
BuildingModel(BuildingEnergy.X[:BuildingEnergy.num_train]).cpu().numpy().flatten()

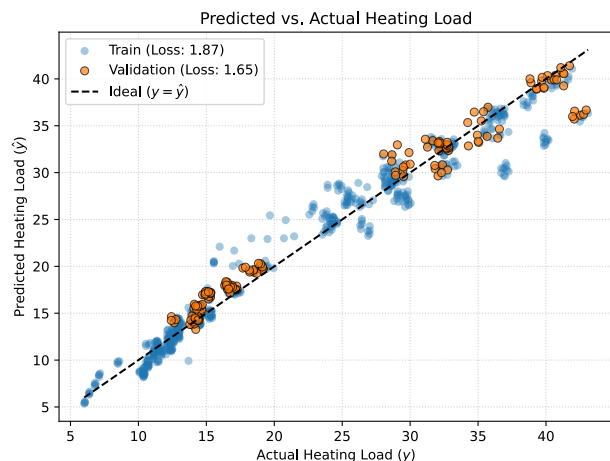
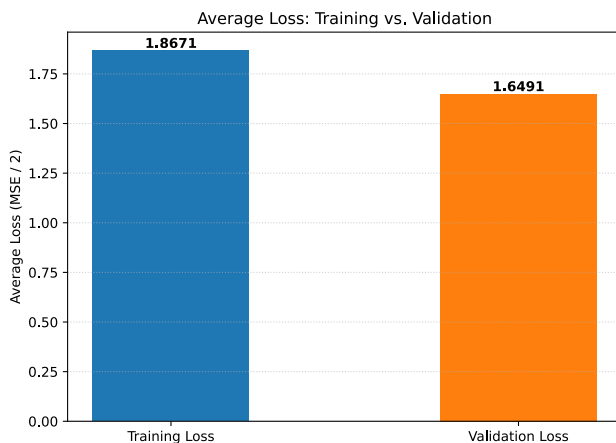
y_val_actual =
BuildingEnergy.y[BuildingEnergy.num_train:].cpu().numpy().flatten()
y_val_pred =
BuildingModel(BuildingEnergy.X[BuildingEnergy.num_train:]).cpu().numpy().flatten()
axes[1].scatter(y_train_actual, y_train_pred, color='tab:blue', alpha=0.4,
label=f'Train (Loss: {train_loss:.2f})', edgecolors='none')
axes[1].scatter(y_val_actual, y_val_pred, color='tab:orange', alpha=0.7,
label=f'Validation (Loss: {val_loss:.2f})', edgecolors='k', linewidths=0.5)
# 45-degree reference line (Ideal  $\hat{y} = y$ )
min_val = min(y_train_actual.min(), y_val_actual.min())
max_val = max(y_train_actual.max(), y_val_actual.max())
axes[1].plot([min_val, max_val], [min_val, max_val], 'k--', lw=1.5,
label='Ideal ( $y = \hat{y}$ )')
axes[1].set_xlabel('Actual Heating Load ($y$)')
axes[1].set_ylabel('Predicted Heating Load ($\hat{y}$)')
axes[1].set_title('Predicted vs. Actual Heating Load')
axes[1].legend()
axes[1].grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
# YOUR CODE ENDS HERE

```

```

slice(0, 614, None)
slice(614, None, None)
Training Loss (MSE / 2): 1.8671
Validation Loss (MSE / 2): 1.6491

```



- Q4.2):** In the code block below, instantiate and train variations of your `BuildingEnergy` data loader and `BuildingModel` model to explore what hyperparameters lead to the lowest `average_loss()` when calculated on the validation set. In the response below, briefly explain your approach and describe your results.

- **A4.2):**
- **Q4.3):** In the code block below, print or visualize the values of $\mathbf{w}$ from our `BuildingModel`. Which weight(s) are the largest? Connect your answer back to your answer to Q1.1. Note that our weights were initialized as a tensor of size `[num_inputs, 1]`, and you might need to `squeeze()` the values to plot them.
- **A4.3):** It looks like weight 8 is the largest by a significant margin. This is likely because the heating and cooling loads had the highest correlation which we saw in Q1.1 when we graphed it and they had very similar curves.

For our LinearRegression_v1 class, the weights are stored as .w. Later, we might want to build a helper function for this part.

YOUR CODE STARTS HERE

```
def get_weights(BuildingModel):
    weights = BuildingModel.w.squeeze().detach().numpy()
    print(weights)
    return weights

def graph_weights(weights):
    if hasattr(weights, 'detach'):
        weights = weights.squeeze().detach().numpy()

    plt.figure(figsize=(8, 4))
    plt.bar(range(len(weights)), weights, color='steelblue',
            edgecolor='black')
    plt.axhline(0, color='gray', linestyle='--', linewidth=0.8)
    plt.xlabel("Feature Index")
    plt.ylabel("Weight Value")
    plt.title("BuildingModel Learned Weights")
    plt.xticks(range(len(weights)))
    plt.grid(axis='y', linestyle=':', alpha=0.6)
    plt.show()
```

```
graph_weights(get_weights(BuildingModel))
```

YOUR CODE ENDS HERE

```
[ -0.3636653  -0.40607494  1.2438941  -1.0145146   2.2533016  -0.10122929
  1.6309988   0.36318663  5.7582154 ]
```



- **Q5.1):** Does the Appliance Energy dataset have any missing data or duplicated rows?
- **A5.1):** No it does not.
- **Q5.2):** Which input features do you expect to have the largest impact on Appliances ? Why? Does this answer make intuitive sense to you?
- **A5.2):** I would expect temperature to have the largest impact in general. This is because many appliances rely on altering temperature (i.e. refrigerators, AC, etc) which have more strain when the outside temperature is closer to its desired temp.
- **Q5.3):** In what ways is this dataset representative of household appliance energy usage in general, and in what ways is it not representative? Relate it back to your own household living situation.
- **A5.3):** It is representative in the fact that it is a large dataset with multiple readings. I would say that its not as representative as it doesn't appear to include location, where location can matter dramatically beyond just weather to culture (some people culturally don't use some appliances like AC or dryers (see Europe)).

```
# Review prior EDA for code to analyze duplicate / missing values, display histograms, and compute correlation heatmap.  
# Note that our dataframe is now called df2 instead of df1, and that our charts may need to be resized for the larger number of features.
```

```
# YOUR CODE STARTS HERE
```

```
# .isna() returns a Boolean if the data is missing, which we then sum up and convert to a Label: Value dict
```

```
print(f"Missing data values: {df2.isna().sum().to_dict()}")
```

```
# .duplicated() returns a Boolean series for any duplicated rows
```

```
print(f"Number of duplicated rows: {df2.duplicated().sum()}")
```

```
numeric_cols = df2.select_dtypes(include=[np.number]).columns
```

```
# .hist() displays a histogram of the dataframe
```

```
_ = df2[numeric_cols].hist(figsize=(20,13),bins=30)
```

```
plt.tight_layout()
```

```
corr = df2[numeric_cols].corr()
```

```
plt.figure(figsize=(14, 11))
```

```
im = plt.imshow(corr.values, cmap="coolwarm", vmin=-1, vmax=1)
```

```
plt.colorbar(im, fraction=0.046, pad=0.04, label="Pearson Correlation")
```

```
plt.xticks(ticks=np.arange(len(corr.columns)), labels=corr.columns,  
rotation=45, ha="right", fontsize=8.5)
```

```
plt.yticks(ticks=np.arange(len(corr.index)), labels=corr.index, fontsize=8.5)
```

```
plt.title("Correlation Heatmap", fontsize=14)
```

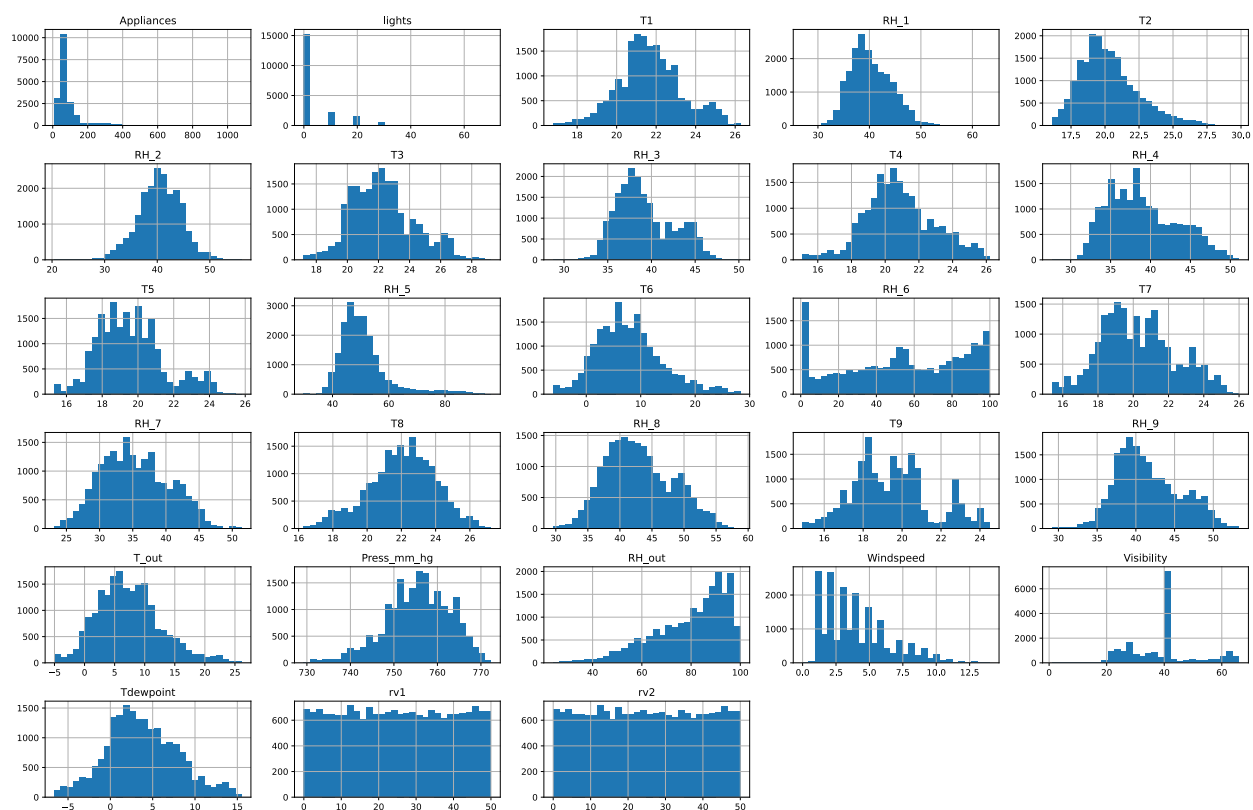
```
plt.tight_layout()
```

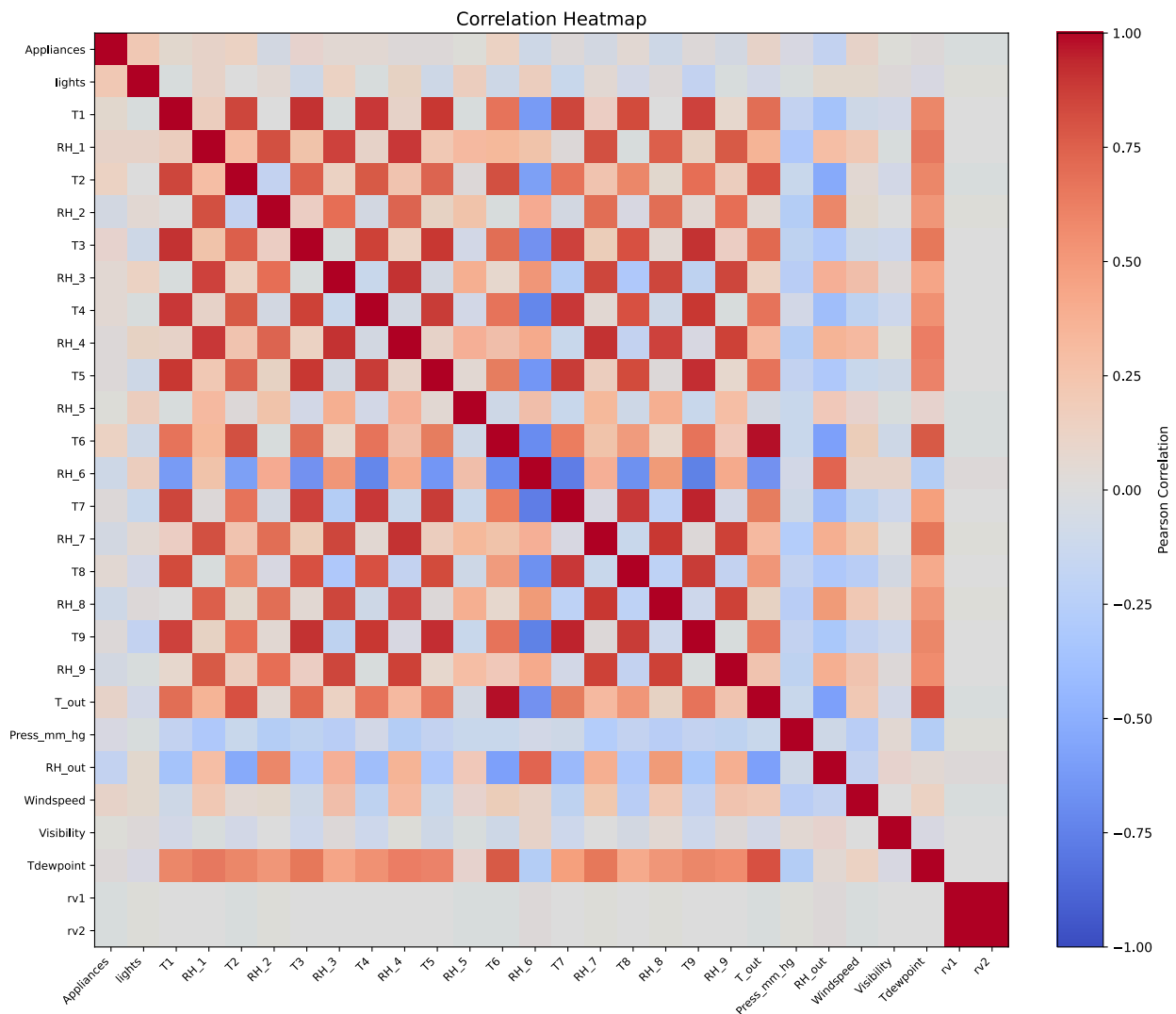
```
plt.show()
```

```
# YOUR CODE ENDS HERE
```

```
Missing data values: {'date': 0, 'Appliances': 0, 'lights': 0, 'T1': 0,  
'RH_1': 0, 'T2': 0, 'RH_2': 0, 'T3': 0, 'RH_3': 0, 'T4': 0, 'RH_4': 0, 'T5':  
0, 'RH_5': 0, 'T6': 0, 'RH_6': 0, 'T7': 0, 'RH_7': 0, 'T8': 0, 'RH_8': 0,  
'T9': 0, 'RH_9': 0, 'T_out': 0, 'Press_mm_hg': 0, 'RH_out': 0, 'Windspeed': 0,  
'Visibility': 0, 'Tdewpoint': 0, 'rv1': 0, 'rv2': 0}
```

```
Number of duplicated rows: 0
```





- **Q5.4):** For any given 2 week period in the dataset (you can use the first two weeks), is there a clear distinction between weekday and weekend appliance energy usage? Can you think of a reason this might be the case?
- **A5.4):** It looks like there is lower appliance usage on weekends compared to weekdays. This surprised us as we thought people would spend more time at home and do more chores like laundry. But it makes sense that people do more work on weekdays, likely in the evening.
- **Q5.5):** How does the outdoor temperature and humidity in May change over the course of the day? Does your result make sense?
- **A5.5):** They appear to have an inverse relationship where as the temperature increases humidity decreases. This makes sense as when the air temperature

increases it is able to hold more water (thus humidity drops) without more water evaporating.

```
df2['day_of_week'] = df2['date'].dt.day_name() # Extract day of the week from
the 'date' column
first_two_weeks = df2[df2['date'] < (df2['date'].min() +
pd.Timedelta(days=14))]

# YOUR CODE STARTS HERE
# Separate weekdays and weekends
first_two_weeks = first_two_weeks.copy()
first_two_weeks["is_weekend"] = first_two_weeks["date"].dt.dayofweek >= 5

weekday_appliances = first_two_weeks.loc[
    ~first_two_weeks["is_weekend"], "Appliances"
]
weekend_appliances = first_two_weeks.loc[
    first_two_weeks["is_weekend"], "Appliances"
]

# Use common bins so the distributions are directly comparable
bins = np.linspace(
    first_two_weeks["Appliances"].min(),
    first_two_weeks["Appliances"].max(),
    25
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

axes[0].hist(weekday_appliances, bins=bins, color="steelblue",
    edgecolor="black", alpha=0.75)
axes[0].set_title("Weekday Appliance Usage")
axes[0].set_xlabel("Appliances Energy Usage (Wh)")
axes[0].set_ylabel("Frequency")
axes[0].grid(axis="y", linestyle=":", alpha=0.5)

axes[1].hist(weekend_appliances, bins=bins, color="darkorange",
    edgecolor="black", alpha=0.75)
axes[1].set_title("Weekend Appliance Usage")
axes[1].set_xlabel("Appliances Energy Usage (Wh)")
axes[1].grid(axis="y", linestyle=":", alpha=0.5)

plt.suptitle("Appliance Usage During the First Two Weeks", fontsize=14)
plt.tight_layout()
plt.show()

# Second graph

# Filter data to May
may_data = df2[df2["date"].dt.month == 5].copy()
```

```

# Extract hour of day
may_data["hour"] = may_data["date"].dt.hour

# Calculate average outdoor temperature and humidity for each hour
hourly_may = may_data.groupby("hour")[["T_out", "RH_out"]].mean()

# Create two graphs
fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

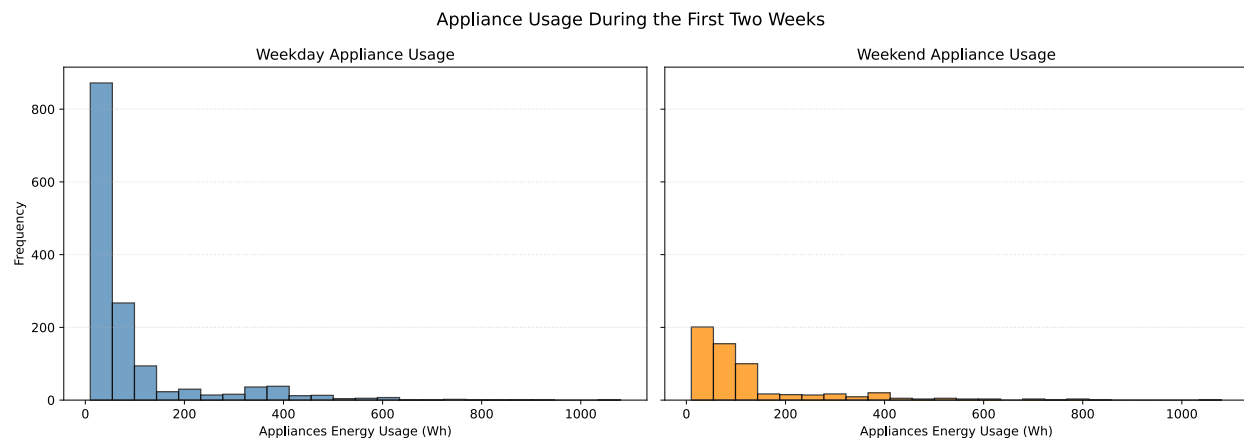
axes[0].plot(hourly_may.index, hourly_may["T_out"],
             marker="o", color="tab:red")
axes[0].set_title("Average Outdoor Temperature by Hour in May")
axes[0].set_ylabel("Temperature (°C)")
axes[0].grid(True, linestyle=":", alpha=0.6)

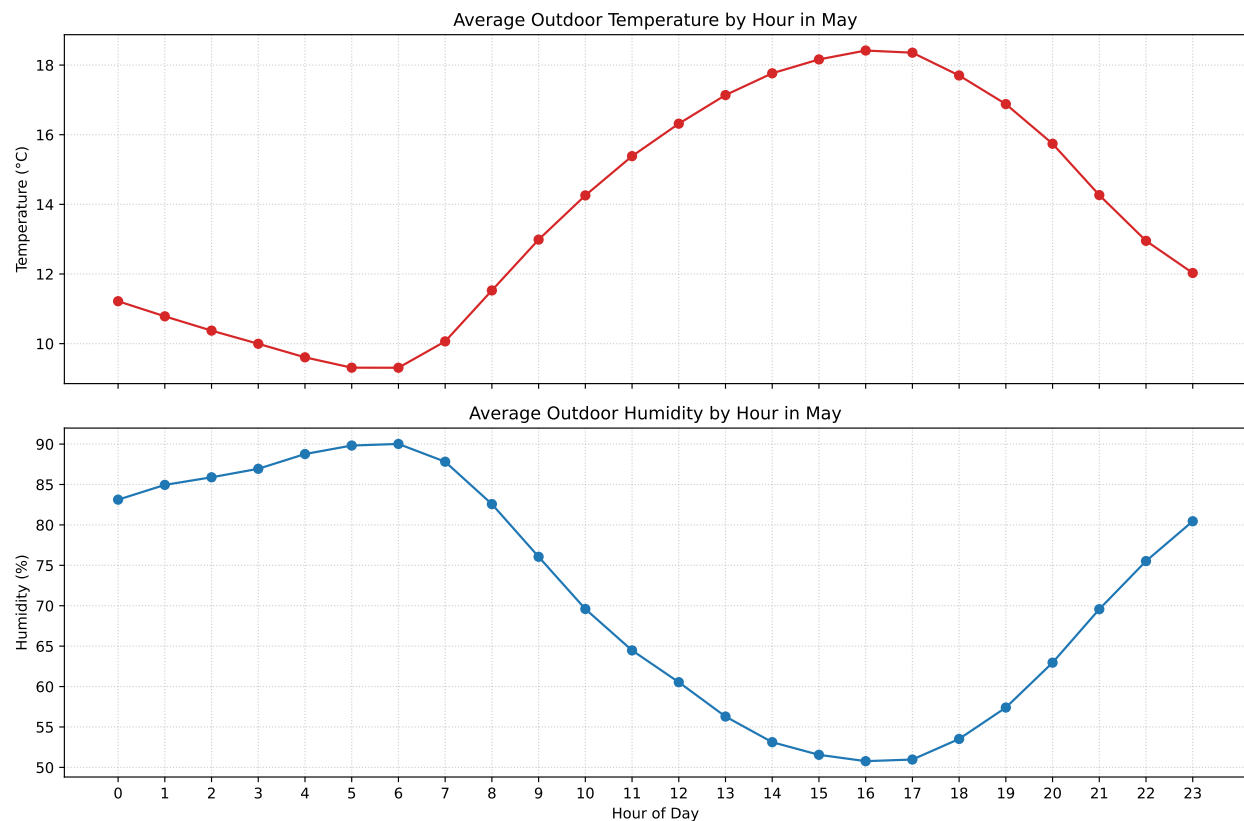
axes[1].plot(hourly_may.index, hourly_may["RH_out"],
             marker="o", color="tab:blue")
axes[1].set_title("Average Outdoor Humidity by Hour in May")
axes[1].set_xlabel("Hour of Day")
axes[1].set_ylabel("Humidity (%)")
axes[1].set_xticks(range(0, 24))
axes[1].grid(True, linestyle=":", alpha=0.6)

plt.tight_layout()
plt.show()

# YOUR CODE ENDS HERE

```





```

class ApplianceEnergyData(d2l.DataModule):  #@save
    """Appliance Energy data for linear regression."""
    def __init__(self, appliance_energy_df, train_val_split=0.8,
batch_size=32, log_transform_target=False):
        super().__init__()
        self.save_hyperparameters()
        # Process energy_df similar as in part 1 when it was called df1
        #print(building_energy_df.info)
        # The resulting self.X and self.y need to be torch.tensors, so be sure
        to convert from pandas DataFrame to numpy array to torch tensor
        # .values gets the underlying numpy array from the DataFrame, and
        .to(torch.float32) converts to float32 tensor
        # example: tensor_vals =
        torch.tensor(dataframe[column_name].values).to(torch.float32)

        y_raw = appliance_energy_df[['Appliances']]

        # Keep only numeric input columns; this removes date
        X_raw = appliance_energy_df.drop(columns=['Appliances'])
        X_raw = X_raw.select_dtypes(include='number')
        means = X_raw.mean()
        stds = X_raw.std(ddof=0)
        safe_stds = stds.replace(0, 1.0)

        X_scaled = (X_raw - means) / safe_stds
        self.X = torch.tensor(X_scaled.to_numpy(), dtype=torch.float32)
        if(log_transform_target):
            y_raw = np.log1p(appliance_energy_df[['Appliances']])

```

```

self.y = torch.tensor(y_raw.to_numpy(), dtype=torch.float32)

self.data = torch.cat((self.X, self.y), dim=1)

total = len(self.data)
self.num_train = int(total * train_val_split)
self.num_val = total - self.num_train
print(self.num_train)

# Our get_dataloader method can be the same as before
def get_dataloader(self, train):
    i = slice(0, self.num_train) if train else slice(self.num_train, None)
    return self.get_tensorloader((self.X, self.y), train, i)

def train_dataloader(self):
    return get_dataloader(self, True)

def val_dataloader(self):
    return get_dataloader(self, False)

```

- **Q5.6):** Plot a visualization of any of the input features returned after calling `.train_dataloader()`. Compare to the previously plotted visualization of the corresponding feature from the initial exploratory data analysis. The graphs should not look identical. Explain two key differences between the visualizations and their cause.
- **A5.6):** The main differences are the values, which are normalized and thus centered around zero and scaled. The other main difference is that the curve itself appears to be much more random, which makes sense as we are selecting a small random batch rather than the whole dataset. One batch is not going to be a perfect representation of the whole dataset.

```

appliance_energy_df = pd.read_csv('data/appliance_energy_data.csv')

# YOUR CODE STARTS HERE
ApplianceData = ApplianceEnergyData(appliance_energy_df=appliance_energy_df,
train_val_split=0.85)

X_batch, y_batch = next(iter(ApplianceData.train_dataloader()))

# Plot the batches
feature_names = (
    appliance_energy_df
    .drop(columns=['Appliances'])
    .select_dtypes(include='number')
    .columns
)

feature = 'T1'

```

```

feature_idx = list(feature_names).index(feature)

# Convert tensors to NumPy arrays
x_values = X_batch[:, feature_idx].detach().cpu().numpy()
y_values = y_batch.squeeze().detach().cpu().numpy()

# Visualize the batch
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].hist(x_values, bins=10, edgecolor='black', color='steelblue')
axes[0].set_title(f'Scaled {feature} Values in One Batch')
axes[0].set_xlabel(f'Scaled {feature}')
axes[0].set_ylabel('Frequency')

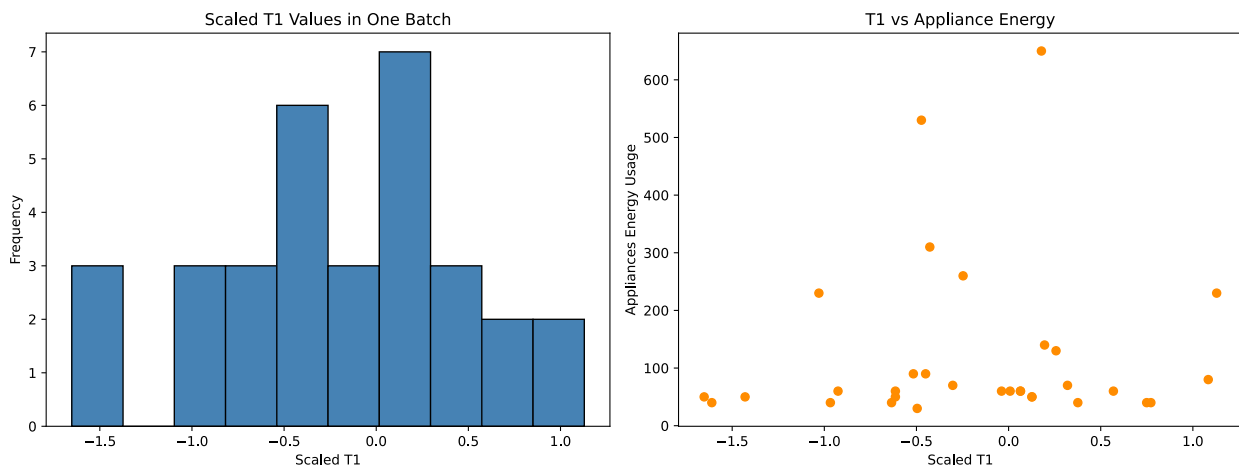
axes[1].scatter(x_values, y_values, color='darkorange')
axes[1].set_title(f'{feature} vs Appliance Energy')
axes[1].set_xlabel(f'Scaled {feature}')
axes[1].set_ylabel('Appliances Energy Usage')

plt.tight_layout()
plt.show()
# YOUR CODE ENDS HERE

```

16774

slice(0, 16774, None)



- **Q5.7):** Like we did before, calculate the average loss for the training and validation data using function `average_loss()`. How does this number compare to the previous regression example? Did we successfully train the model?
- **A5.7):** We originally had a loss of about ~4600 on training and ~3700 for validation for this model, this was because while we were normalizing the input features but the output y was not normalized. There is still a large gap between the training and validation even though the magnitude is very off. This is not a

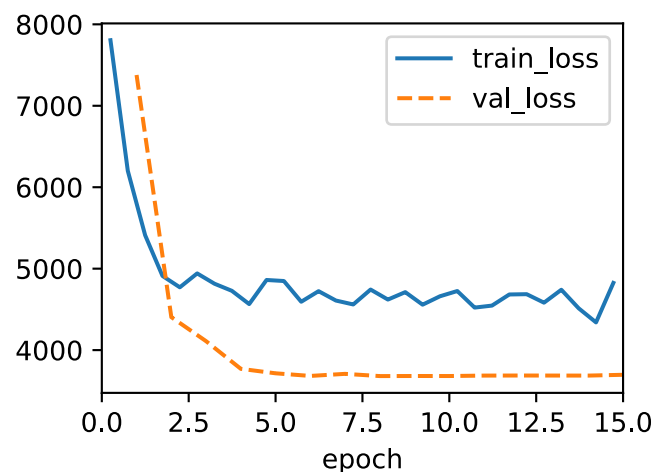
well trained model, despite it appearing to generalize against the validation it does poorly on the training.

```
# YOUR CODE STARTS HERE
ApplianceModel = LinearRegression_v1(X_batch.shape[1], lr=0.002)
ApplianceModelTrainer = d2l.Trainer(max_epochs=15)
ApplianceModelTrainer.fit(ApplianceModel, ApplianceData)

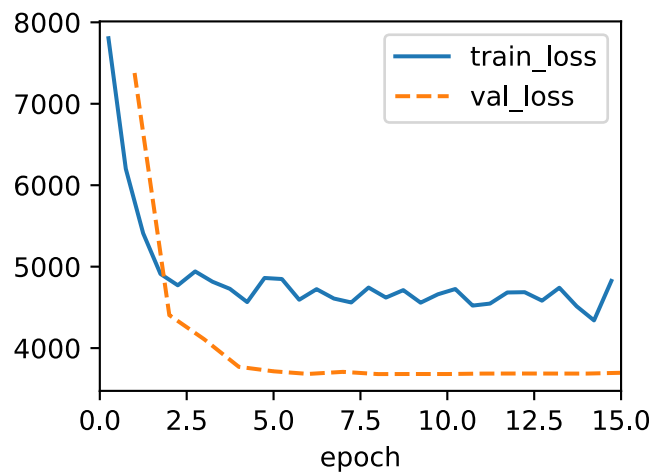
val_loss = average_loss(ApplianceModel, ApplianceData.val_dataloader(),
ApplianceModel.loss)
train_loss = average_loss(ApplianceModel, ApplianceData.train_dataloader(),
ApplianceModel.loss)

print(f"Validation loss -> {val_loss}")
print(f"Train_loss -> {train_loss}")
# YOUR CODE ENDS HERE
```

```
slice(0, 16774, None)
slice(16774, None, None)
```



```
slice(16774, None, None)
slice(0, 16774, None)
Validation loss -> 3680.352269980119
Train_loss -> 4596.749940856956
```



- **Q6.1):** Describe your implementation changes that you made to improve your model, and how your training and validation loss have changed.
- **A6.1):** We added the log function to the Y input data to normalize it, which can be seen below. Now the validation and training loss are significantly closer and smaller from ~4900 and ~3900 down to 0.162 and 0.126 respectively. This is the one improvement we implemented.

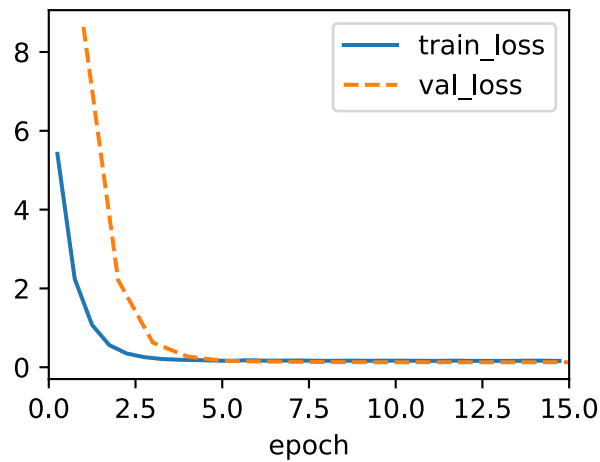
```
# YOUR CODE STARTS HERE
ApplianceData = ApplianceEnergyData(appliance_energy_df=appliance_energy_df,
train_val_split=0.85, log_transform_target=True)

X_batch, y_batch = next(iter(ApplianceData.train_dataloader()))
ApplianceModel = LinearRegression_v1(X_batch.shape[1], lr=0.002)
ApplianceModelTrainer = d2l.Trainer(max_epochs=15)
ApplianceModelTrainer.fit(ApplianceModel, ApplianceData)

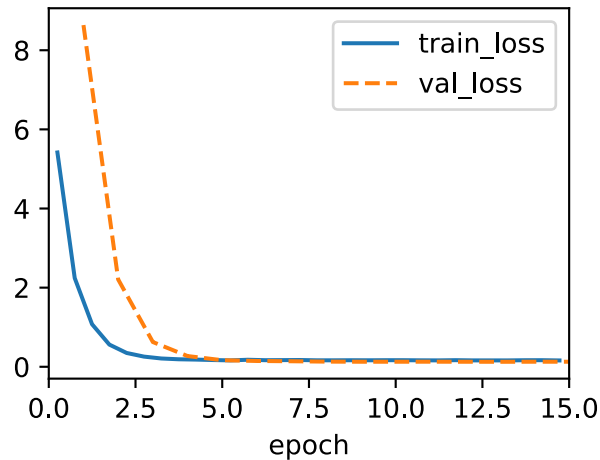
val_loss = average_loss(ApplianceModel, ApplianceData.val_dataloader(),
ApplianceModel.loss)
train_loss = average_loss(ApplianceModel, ApplianceData.train_dataloader(),
ApplianceModel.loss)

print(f"Validation loss -> {val_loss}")
print(f"Train_loss -> {train_loss}")
# YOUR CODE ENDS HERE
```

```
16774
slice(0, 16774, None)
slice(0, 16774, None)
slice(16774, None, None)
```



```
slice(16774, None, None)
slice(0, 16774, None)
Validation loss -> 0.12712195595648995
Train_loss -> 0.16217523223117636
```



- **Q7.1):** Why does it not make sense to scale / normalize our new `proto_TCP` , `proto_ICMP` , and `proto_UDP` features?
- **A7.1):** Because they are binary, 0 or 1 and exclusive (only one can be 1). There isn't really way to scale something like that.
- **Q7.2):** Mathematically, why does taking the `log()` of our features turn a multiplicative relationship into an additive one?
- **A7.2):** It is because on a logarithmic scale increasing by 10x would effectively be the same as adding 1 and 100x is adding 2 assuming the base was 10. So rather than multiplying its adding it in relation to the exponent/base of the log.

```

class CyberFlowData(d2l.DataModule):  #@save
    """Cyber Flow data for linear regression."""
    def __init__(self, cyber_flow_df, train_val_split=0.8, batch_size=32):
        super().__init__()
        self.save_hyperparameters()

        cyber_flow_df_encoded = pd.get_dummies(cyber_flow_df, columns=
['protocol'], prefix='proto', dtype=int)
        cols_to_scale = ['packet_count', 'avg_packet_size', 'rtt_ms',
'packet_mult_est'] # You will need to compute packet_mult_est first
        protocol_cols = ["proto_ICMP", "proto_TCP", "proto_UDP"]

        # goal -> predict bytes_per_flow
        # first calculate packet_mult_est (non scaled)
        cyber_flow_df_encoded['packet_mult_est'] =
cyber_flow_df_encoded['packet_count'] *
cyber_flow_df_encoded['avg_packet_size']

        # now normalize X and y
        y_raw = cyber_flow_df_encoded[['bytes_per_flow']]

        X_raw = cyber_flow_df_encoded[cols_to_scale + protocol_cols]
        means = X_raw[cols_to_scale].mean()
        stds = X_raw[cols_to_scale].std(ddof=0).replace(0, 1.0)
        #safe_stds = stds.replace(0, 1.0)

        # shift around zero
        X_scaled = X_raw.copy()
        X_scaled[cols_to_scale] = (X_raw[cols_to_scale] - means) / stds

        self.X = torch.tensor(X_scaled.to_numpy(), dtype=torch.float32)

        # norm y
        y_mean = y_raw.mean()
        y_std = y_raw.std(ddof=0).replace(0, 1.0)
        y_scaled = (y_raw - y_mean) / y_std
        self.y = torch.tensor(y_scaled.to_numpy(), dtype=torch.float32)

        self.data = torch.cat((self.X, self.y), dim=1)

        total = len(self.data)
        self.num_train = int(total * train_val_split)
        self.num_val = total - self.num_train

    def train_dataloader(self):
        return get_dataloader(self, True)

    def val_dataloader(self):
        return get_dataloader(self, False)

    # Our get_dataloader method can be the same as before
    def get_dataloader(self, train):

```

```
i = slice(0, self.num_train) if train else slice(self.num_train, None)
return self.get_tensorloader((self.X, self.y), train, i)
```

- **Q7.3):** Plot a visualization of one of the scaled input features returned after calling `.train_data_loader()` (the second feature `X[:,1]` is appropriate to select). Compare to a visualization of the one-hot encoded feature (the fifth feature is `proto_ICMP`). Describe your results - do they seem reasonable?
- **A7.3):** Yes, the average packet size (second feature) is centered around 0. The one-hot encoded feature seems reasonable as well, given that in the data we likely don't have an equal amount of ICMP, TCP, and UDP types. It is still around ~15% of all packets (note this number changes depending on the batch).

```
cyber_flow_df = pd.read_csv('data/cyber_flows_data.csv')

# YOUR CODE STARTS HERE
CyberData = CyberFlowData(cyber_flow_df)
X, y = next(iter(CyberData.train_data_loader()))

# Convert the selected tensor columns to NumPy arrays
scaled_feature = X[:, 1].detach().cpu().numpy()
proto_icmp = X[:, 4].detach().cpu().numpy()

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

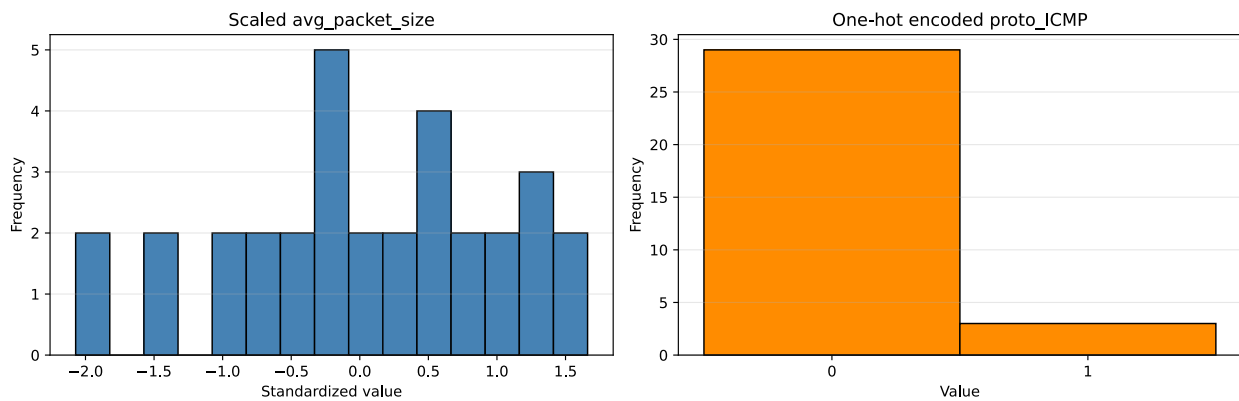
# Second feature: avg_packet_size
axes[0].hist(scaled_feature, bins=15, edgecolor="black", color="steelblue")
axes[0].set_title("Scaled avg_packet_size")
axes[0].set_xlabel("Standardized value")
axes[0].set_ylabel("Frequency")
axes[0].grid(axis="y", alpha=0.3)

# Fifth feature: proto_ICMP
axes[1].hist(proto_icmp, bins=[-0.5, 0.5, 1.5],
             edgecolor="black", color="darkorange")
axes[1].set_title("One-hot encoded proto_ICMP")
axes[1].set_xlabel("Value")
axes[1].set_ylabel("Frequency")
axes[1].set_xticks([0, 1])
axes[1].grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

# YOUR CODE ENDS HERE

slice(0, 2400, None)
```



- **Q7.4):** Review the documentation for [LazyLinear](#) and [LazyModuleMixin](#). In your own words, explain how this "lazy" capability is implemented.
- **A7.4):** It is lazy because the model parameters are not initialized until the first call to `forward` is done.

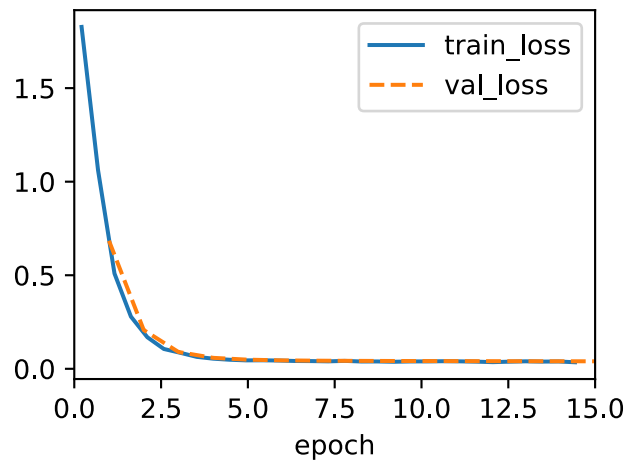
```
# YOUR CODE STARTS HERE
# Set our data
CyberData = CyberFlowData(cyber_flow_df, train_val_split=0.80, batch_size=128)
X, y = next(iter(CyberData.train_dataloader()))
# build the model
CyberModel = LinearRegression_v2(lr=0.01)
#build the trainer
CyberModelTrainer = d2l.Trainer(max_epochs=15)
# training run
CyberModelTrainer.fit(CyberModel, CyberData)

#Get training and validation loss
val_loss = average_loss(CyberModel, CyberData.val_dataloader(),
CyberModel.loss)
train_loss = average_loss(CyberModel, CyberData.train_dataloader(),
CyberModel.loss)

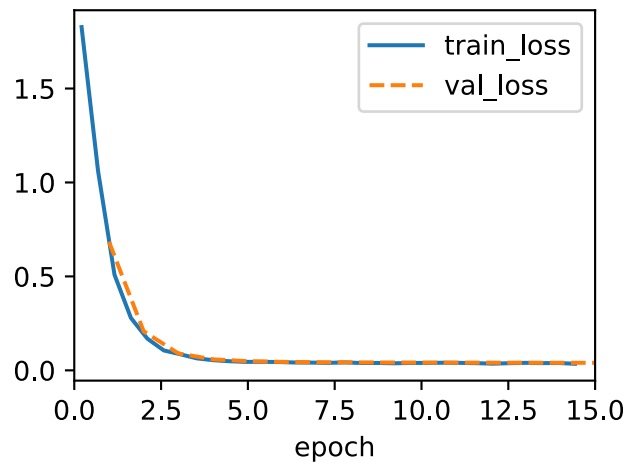
print(f"Validation loss -> {val_loss}")
print(f"Train_loss -> {train_loss}")

# YOUR CODE ENDS HERE

slice(0, 2400, None)
slice(0, 2400, None)
slice(2400, None, None)
```



```
slice(2400, None, None)
slice(0, 2400, None)
Validation loss -> 0.040698439578215284
Train_loss -> 0.03751918916900953
```



Prelab Questions

- **Q0.1):** Why is it important to perform an Exploratory Data Analysis (EDA) before attempting Linear Regression? What can EDA tell us about our data that might influence how we perform regression?
- **A0.1):** EDA can tell us very important information that we might not know if we just jump right in. Particularly with visualizations it can tell us what amount of each data we have, any gaps in the data, cluster density and distance, and depending on those shapes it would change how we implement regression.
- **Q0.2):** As you review the lab document, you will come across three different methods to compute the forward pass, which for a simple linear regression is just the matrix-vector product of $\mathbf{X}$, $\mathbf{w}$, and b . Briefly describe each method.

- **A0.2):**

Method 1: Basic dot product for *mostly* linear data.

$$\hat{y} = w_1x_1 + \dots + w_dx_d + b.$$

Method 2: Sum of dot products across all input embeddings and weights + bias

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

Method 3: A final form, the best representation, as you can treat it as a regular matrix multiplication + bias. Much nicer for parallelism even through the underlying dot product operation is the same.

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b.$$

All of these might look very different but they are all computing the same matrix-vector product just with different syntax and math implementation. As far as simplistity for understanding method 1 is the easiest to get an initial grasp on. But the last mthod is by far the nicest to actually implement with libraries like torch that do matrix-vector products in one line.

- **Q0.3):** If you observe very large gradients during training, what could be the cause, and what are some approaches you could use to address it?
- **A0.3):** If the gradients are very large during training there can be many causes. Arguably most likely it is not properly normalizing the input features, which should have been caught in EDA probably. That can be addressed with normalizing the inputs when calculating the gradients pre-training. Its also possible that the hyperparameters (epochs, learning rate, etc) are not initialized properly although those would more so impact training rather than the gradients directly.